

GET /cloud/impinjGen2X

The read side of the Impinj Gen2X endpoint. **This folder previously held only an empty 1-byte response** - that capture was correct at the time but misleading on its own, so it is now kept alongside a real one with the cause explained.

Readers	10.233.48.36 (FXR60) and 10.233.48.49 (FXR90), both app 5.0.7
Dates	2026-09-05 (empty capture), 2026-09-08 (re-tested)
Result	[OK] 200 in both states - body depends on whether Gen2X has ever been configured

Why the original capture looked empty

`GET /cloud/impinjGen2X` echoes stored Gen2X configuration. On 2026-09-05 no Gen2X feature had ever been set on that reader, so it returned a bare newline:

	2026-09-05 (never configured)	2026-09-08 (after a PUT)
Status	200	200
Size	1 byte	28 bytes
Body	0x0a - a bare newline	{"fastID":{"enabled":false}}
Valid JSON?	[X] no	[OK] yes

Verified identical on **both** readers on 2026-09-08:

10.233.48.36	FXR60	5.0.7	->	200, 28 bytes, {"fastID":{"enabled":false}}
10.233.48.49	FXR90	5.0.7	->	200, 28 bytes, {"fastID":{"enabled":false}}

So the endpoint was never broken - it had nothing to report. The empty state can only be seen on a reader that has never had Gen2X configured, because **the config is sticky**:

PUT /cloud/impinjGen2X {}
-> 422 {"code":1,"message":"Invalid payload for set_impinjGen2X api"}

There is no documented way back to empty.

[BUG] Two findings worth reporting

1. The empty response is not valid JSON. The spec documents the 200 response as an `impinjGen2X` object. A bare `0x0a` is neither an object nor `{}`, and `json.loads()` on it raises `Expecting value: line 1 column 1`. A client hitting a factory-fresh reader gets a parse error, not an empty config. Returning `{}` would fix it.

2. `GET` only ever reflects the last feature configured. `PUT` is wholesale, not a merge - proven directly:

PUT {"tagFocus":{"enabled":true}}	->	200	
GET	->	{"tagFocus":{"enabled":true}}	
PUT {"fastID":{"enabled":false}}	->	200	
GET	->	{"fastID":{"enabled":false}}	<- tagFocus gone

The stored `tagFocus` object is replaced, not kept. This follows from the reader's own one-feature-per-request rule, but the consequence is worth stating: `GET /cloud/impinjGen2X` cannot be used to read back the reader's overall Gen2X state`. A client that sets `tagFocus`, then later sets `fastID`, has no API-visible record that `tagFocus` was ever

configured.

Folder contents

```
cloud-impinjGen2X/
|-- README.md
|-- 01-GET-never-configured-EMPTY/
|-- 02-GET-after-configured-JSON/
`-- 03-GET-reflects-last-feature-only/

<- this file
the original Sep-5 capture, 1 byte
proper JSON, both readers
PUT is wholesale, not a merge
```

Each holds **request body/request.json** and **response body/** with the verbatim response, **http_status.txt** and **verification.txt**. The original 1-byte capture is preserved unmodified in **01-**.

Related - the write side and feature testing

Where	What
../cloud-impinjGen2X-PUT/all-features-VERIFIED-507/	All four features verified working on Monza 4i - fastID, tagFocus, tagQuieting, tagProtect
../cloud-impinjGen2X-PUT/tagprotect-REVERIFIED-507/	TagProtect re-verified on 5.0.7, no regression
../cloud-impinjGen2X-PUT/tagprotect-CONFIRMED-monza4i/	The original TagProtect proof on 5.0.4
../cloud-impinjGen2X-PUT/v3-all-examples/	All 12 spec examples tested
../cloud-impinjGen2X-PUT/SET_ACCESS_PASSWORD.md	How to assign the 32-bit access password